

Guardian Cloud Project

지능화된 웹 공격과 대규모 트래픽 폭주에 대응하는 GCP 클라우드 네이티브 DevSecOps 인프라
구축

Team. 강세권 8번출구

Members. 김용균, 서희정, 윤승호, 이지희, 허상준

KT Cloud 인프라 2기

팀 구성 및 역할 (TEAM & ROLES)

이름	포지션	주도적 참여 부분 (담당 업무)
김용균	Team Leader / Infra	VPC 및 Subnet 설계, 클라우드 아키텍처 총괄 및 CI/CD 파이프라인 구성
서희정	Backend Engineer	Service A 기능 구현 및 부하테스트, ip 기반 차단 시나리오 설계
윤승호	Backend Engineer	Service B 기능 구현 및 SQL 인젝션, Brute force 시나리오 설계
이지희	Frontend / Monitoring	Service A, B UI 구현, 모니터링 시스템 구축, 대시보드 설계
허상준	Cloud Security Engineer	보안 정책 수립, SIEM 기반 탐지 & 알람 체계 및 자산 보호 자동화 구축

목차 (TABLE OF CONTENTS)

1. 프로젝트 개요

기술 스택, 주제 선정 의도,
전체 아키텍처 설계도

2. 네트워킹 및 인프라

네트워킹 및 VPC 설계 및 흐름도,
CI/CD, 인프라 프로비저닝

3. 시나리오 A, B

각 시나리오 별 기능 & 구조,
시나리오 시연 영상

4. 백업/ 로그 보관 정책 및 예상 비용

백업 / 로깅 정책, 사용한 GCP 서비스 스펙 및
월간 예상비용 산정

5. 자체 평가

각 파트별 트러블 슈팅, 핵심성과 및 추후 개선방안

1. Project Overview

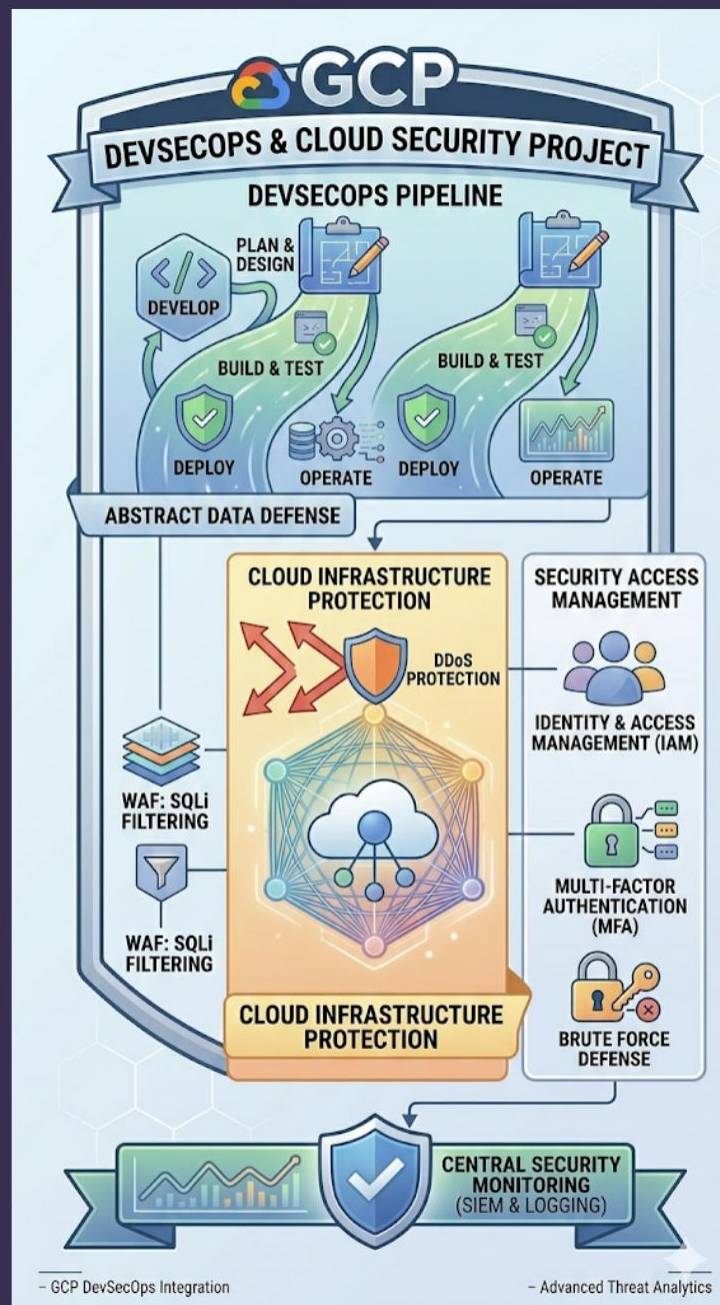
주제 선정 배경 및 기획 의도

최근 클라우드 환경 내 설정 오류를 이용한 보안 사고와 고도화된 L7 계층 공격이 급증함에 따라, 기능 구현을 넘어 실시간 탐지(SIEM), 자산 보호(Secret Management), 그리고 서비스 연속성(Load Balancing & Circuit Breaker)이 통합된 내구성 있는 보안 인프라를 구축하였습니다.

나아가 실제 공격 및 부하 시나리오를 바탕으로 인프라의 성능을 검증하고 평가하는 것을 본 프로젝트의 핵심 목표로 설정하였습니다

차별화된 특화 포인트

- ☞ 단순 구현에 그치지 않고 시나리오 별 대응 방안 과 로깅/백업 정책을 수립 하였습니다
- ⚡ 트래픽 가용성 및 탄력적 서비스 제어를 구현하여 서비스의 연속성을 보장하였습니다
- 🛡️ 보안 내재화 및 실시간 대응체계를 구축하였습니다



사용한 기술 스택 (TECH STACK)

클라우드 벤더 & OS



인프라 프로비저닝 & CI/CD



스토리지 & DB



마이크로 서비스



모니터링 & 로깅



보안

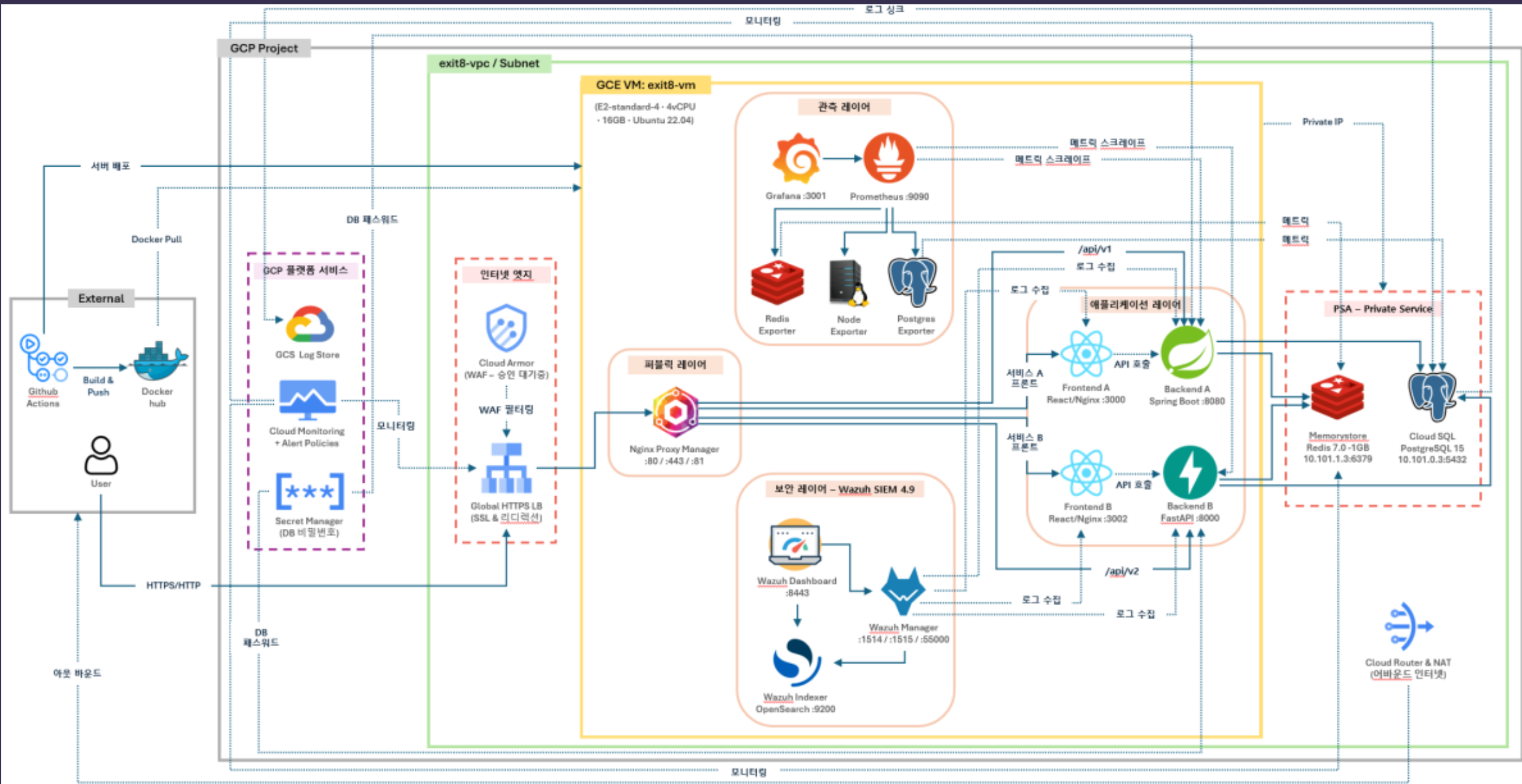


네트워킹



트래픽 관리





2. Network & Infra Provisioning

외부 로드 밸런서 와 보안 (LB & WAF)

GCP Global Load Balancer

전 세계 어디서든 단일 IP로 접속하며, SSL 복호화를 처리해 서버 부하를 줄이고, 가장 가까운 서버로 트래픽을 배분하는 L7 로드 밸런서

- ✔ **L7 HTTPS LB: Anycast** 글로벌 단일 IP 및 SSL 종단 처리
- ✔ **Automation:** HTTP → HTTPS 301 자동 리디렉션 지원
- ✔ **Health Check:** VM:8081 /health 경로를 통한 백엔드 감시
- ✔ **Firewall:** :80 / :443 / :8081 / :22 (ssh) / 내부 트래픽 포트 허용

exit8-lb-redirect

전역 외부 애플리케이션 부하 분산기

세부정보 모니터링 캐시 무효화

프런트엔드

프로토콜 ↑	IP:포트	인증서	인증서 맵	SSL 정책	네트워크 등급 ?
HTTP	34.128.162.9:80	-			프리미엄

프런트엔드

프로토콜 ↑	IP:포트	인증서	인증서 맵	SSL 정책
HTTPS	34.128.162.9:443	exit8-lb-cert		exit8-lb-ssl-policy

1. exit8-lb-backend

엔드포인트 프로토콜	HTTP
이름이 지정된 포트	http
제한 시간	100초
IP 주소 선택 정책 ?	IPv4만
상태 확인	exit8-lb-health-check

외부 로드 밸런서 와 보안 (LB & WAF)

Cloud Armor

GLB에 결합되어 악성 트래픽과 DDoS 공격을 실시간으로 탐지하고,
허용된 사용자만 내부 서비스에 접근할 수 있게 하는 웹 방화벽(WAF)

- ✔ GLB가 SSL을 풀자마자 Cloud Armor가 검사한 패킷만 내부망으로 전송
- ✔ 보안의 일원화 : SSL 인증서와, 방화벽 규칙, WAF 를 중앙처리 하여 간결함
- ✔ 보안 정책 를 통한 SQLi/XSS/Brute-force 및 Ddos 차단 구현 가능
- ❗ 무료 티어 계정에서는 별도의 승인절차 필요 (현재는 주석처리)

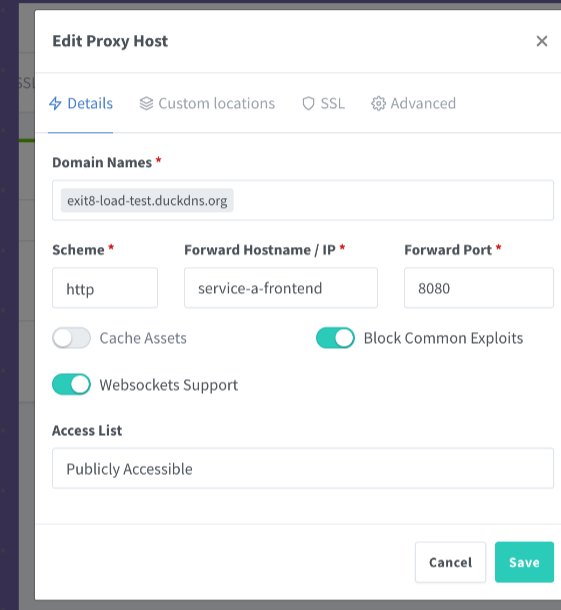
```
resource "google_compute_security_policy" "exit8_security" {  
  # name = "${var.lb_name}-security-policy"  
  # project = var.project_id  
  #  
  # Default rule - allow all (lowest priority)  
  # rule {  
  #   action = "allow"  
  #   priority = "2147483647"  
  #   match {  
  #     versioned_expr = "SRC_IPS_V1"  
  #     config {  
  #       src_ip_ranges = ["*"]  
  #     }  
  #   }  
  #   description = "Default allow rule"  
  # }  
  #  
  # =====  
  # Rate Limiting Rules (Priority 1000-1999)  
  # =====  
  #  
  # 브루트포스 차단: 동일 IP에서 60초당 로그인 시도 10회 초과 시 5분 차단  
  # rule {  
  #   action = "rate_based_ban"  
  #   priority = "1000"  
  #   match {  
  #     versioned_expr = "SRC_IPS_V1"  
  #     config {  
  #       src_ip_ranges = ["*"]  
  #     }  
  #   }  
  #   rate_limit_options {  
  #     rate_limit_threshold {  
  #       count = 10  
  #       interval_sec = 60  
  #     }  
  #     ban_duration_sec = 300  
  #     conform_action = "allow"  
  #     exceed_action = "deny(429)"  
  #     enforce_on_key = "IP"  
  #   }  
  #   description = "Brute force rate limiting - 10 req/60s per IP"  
  # }
```

내부 트래픽 흐름 (NPM)

Nginx Proxy Manager (NPM)

단일 VM 환경에서 복잡한 Nginx 설정 없이, GUI 를 통해
다수 서비스를 URL/포트 기반으로 라우팅 하는 핵심 리버스
프록시

- ✔ **Service Discovery** : 내부 서비스의 '지도'
- ✔ **Internal Security** : 특정 IP 차단, Auth(로그인 창) 추가 적용 용이
- ✔ **SSL Termination** 과 **443 Redirection** 수행하지 않음



These proxy details are available as nginx variables:

- `$server` Forward Hostname / IP
- `$port` Forward Port
- `$forward_scheme` Scheme

Custom Nginx Configuration

```
auth_basic off;  
auth_request off;  
allow all;  
default_type "text/plain";  
root /data/letsencrypt-acme-challenge;  
proxy_read_timeout 100s;  
proxy_send_timeout 100s;  
proxy_connect_timeout 10s;
```

⚠ Please note, that any `add_header` or `set_header` directives added here will not be used by nginx. You will have to add a custom location `/'` and add the header in the custom config there.

Service	Port	Description
Nginx Proxy Manager	80, 443, 81	Reverse Proxy / SSL
Service-A Backend	8080	Spring Boot API (2-Tier Cache)
Service-A Frontend	3000	React + Vite
Service-B Backend	8000	FastAPI API
Service-B Frontend	3002	React + Vite
Prometheus	9090	Metrics Collector
Grafana	3001	Visualization Dashboard
Wazuh Manager	1514, 1515, 55000	SIEM Agent
Wazuh Dashboard	8443	SIEM Web UI

내부 컨테이너 레벨의 네트워크 격리

3-Tier 기반의 네트워크 격리를 통해 컨테이너 레벨 보안을 강화하였습니다.

Network	Containers	Role / Responsibility
frontend	NPM, Service A/B Frontend	외부 트래픽 수신 및 프록시 전달
backend	App, Prometheus, Grafana, Exporters	내부 서비스 간 고속 API 통신
wazuh_internal	Manager, Indexer, Dashboard	SIEM 클러스터 데이터 격리 및 보호

```
prometheus:
  image: prom/prometheus:v2.48.0
  container_name: prometheus
  restart: unless-stopped
  ports:
    - "9090:9090"
  volumes:
    - ./services/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
    - prometheus_data:/prometheus
  command:
    - '--config.file=/etc/prometheus/prometheus.yml'
    - '--storage.tsdb.path=/prometheus'
    - '--storage.tsdb.retention.time=15d'
    - '--storage.tsdb.retention.size=2GB'
    - '--web.enable-lifecycle'
  networks:
    - backend
  logging: *default-logging
  deploy:
    resources:
      limits:
        cpus: '0.5'
        memory: 1G
      reservations:
        memory: 512M
```

내부 DB 망 격리 (PSA)

PSA (Private Service Access)

VPC 피어링 기술을 활용하여 Google 관리 서비스 네트워크와 직접 연결합니다.

- ✓ 핵심 DB 인 Redis 와 PostgreSQL DB 망 분리
- ✓ 공개 IP 없이 프라이빗 IP 로만 통신
- ✓ 인터넷을 경유하지 않는 Google 내부망 트래픽
- ✓ VPC 피어링 예약 IP 대역 (/16) 활용

Service	Private IP	Security Feature
Cloud SQL (PG 15)	10.101.0.3	SSL ENCRYPTED ONLY
Memorystore (Redis)	10.101.1.3	PSA Only Access



인스턴스 > exit8-redis (Exit8 Redis Cache): 연결

연결

기본 엔드포인트

10.101.1.3:6379

승인된 네트워크

exit8-vpc (thinking-orb-485613-k3)

연결 모드

비공개 서비스 액세스

IP 범위

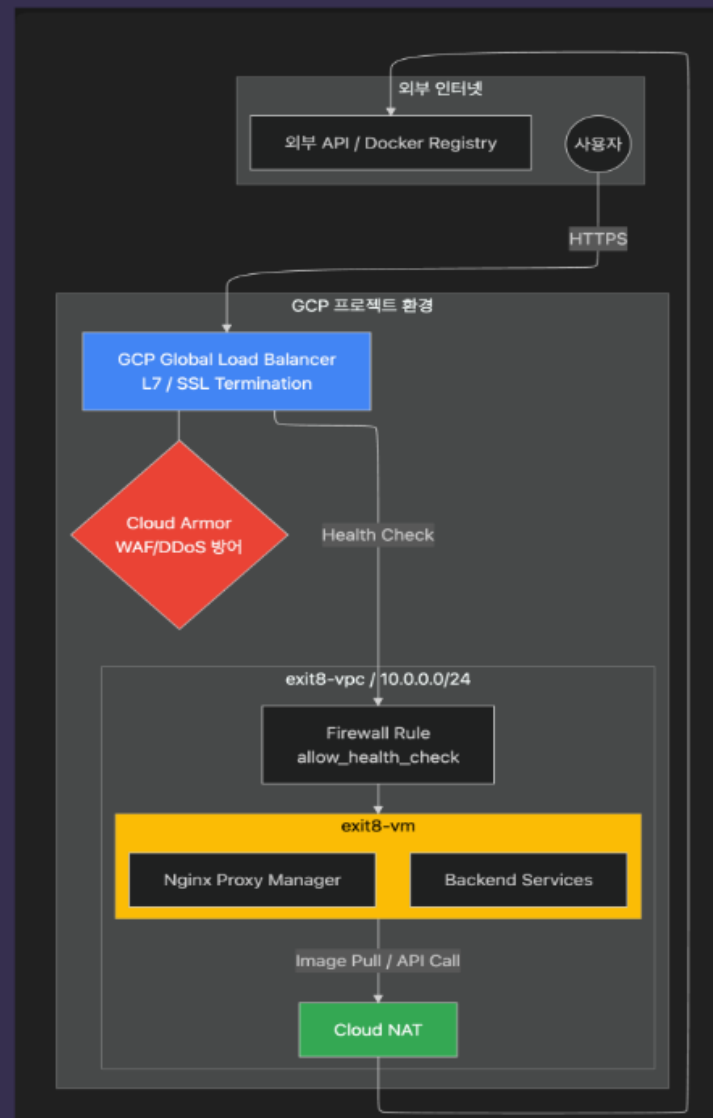
10.101.1.0/29

외부 인터넷 아웃바운드 통신 (Cloud NAT)

Cloud NAT

VM은 외부에서 직접 접속할 수 있는 '공개 IP'가 없지만, 내부에서 **Docker 이미지**를 내려받거나 **외부 API**를 호출할 때는 외부로 나가야 합니다. 이때 Cloud NAT가 VM의 사실 IP를 공인 IP로 변환해 안전하게 밖으로 내보내 줍니다.

- ✔ VM → 외부 인터넷 아웃바운드 통신 (Docker 이미지 pull, 외부 API 호출)
- ✔ 자동 IP 할당(AUTO_ONLY), 100% 로깅 활성화



CI/CD

✓ All checks have passed
6 successful checks

✓

Security Scan / Container Scan - service-a-backend (pull_request)

Successful in 1m

✓

Security Scan / Container Scan - service-a-frontend (pull_request)

Successful in 25s

✓

Security Scan / Container Scan - service-b-backend (pull_request)

Successful in 37s

✓

Security Scan / Container Scan - service-b-frontend (pull_request)

Successful in 29s

✓

Security Scan / Dependency Vulnerability Scan (pull_request)

Successful in 43s

✓

Security Scan / Secret Detection (pull_request)

Successful in 14s

No conflicts with base branch

Security Scan

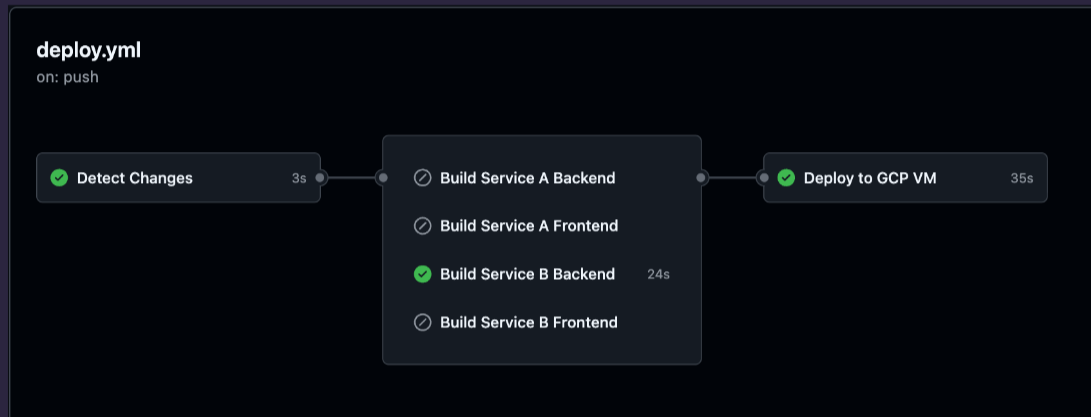
트리거 : PR 생성

Dependency Scan : 소스코드 내에서 의존성 패키지 취약점 탐지

Container Scan : Docker 이미지 내 OS/런타임 취약점 탐지

Secret Scan : 전체 git history 에서 하드코딩된 비밀번호 탐지

** Container Scan 은 4개의 이미지 병렬로 실행



Deploy

트리거 : Main 브랜치 Merge 시

Detect Changes : 변경된 서비스만 선별 (paths-filter)

Build (병렬) : Docker 이미지 build & Docker hub push

Deploy + Health Check : VM 에 ssh 접속 배포 후 컨테이너 헬스 체크

** 동시 배포 방지 (concurrency 설정, 진행 중 취소 불가)

프로비저닝 (Infra Provisioning)

```
cloud_sql.tf
compute.tf
load_balancer.tf
main.tf
memorystore.tf
monitoring.tf
outputs.tf
psa.tf
secrets.tf
subnet.tf
terraform.tfstate
terraform.tfstate.1772258
terraform.tfstate.1772348
terraform.tfstate.backup
terraform.tfvars
```

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:

```
alert_policy_names = {
  "high_cpu" = "Exit8 - High CPU Usage"
  "high_db_connections" = "Exit8 - High DB Connections"
  "high_redis_memory" = "Exit8 - High Redis Memory"
  "lb_5xx_errors" = "Exit8 - High 5xx Error Rate"
}
application_env_vars = <sensitive>
cloud_sql_connection_name = "thinking-orb-485613-k3:asia-northeast3:exit8-postgres"
cloud_sql_database_name = "exit8_app"
cloud_sql_password_secret_id = "projects/280624919718/secret/exit8-db-password/ver"
cloud_sql_private_ip = "10.101.0.3"
cloud_sql_user_name = "exit8_app_user"
estimated_monthly_cost = {
  "cloud_sql" = "~$50/month (db-custom-2-8192 + 100GB SSD)"
  "lb" = "~$20/month (HTTPS LB + Cloud Armor)"
  "redis" = "~$35/month (16GB Basic)"
  "total" = "~$175/month (~₩250,000)"
  "vm" = "~$70/month (e2-standard-4 + 100GB SSD)"
}
grafana_admin_secret_id = "projects/280624919718/secret/exit8-grafana-admin/version"
lb_https_url = "https://34.128.162.9"
lb_ip_address = "34.128.162.9"
monitoring_dashboard_url = "https://console.cloud.google.com/monitoring/dashboards?"
redis_connection_string = "10.101.1.3:6379"
redis_host = "10.101.1.3"
redis_port = 6379
ssh_command = "gcloud compute ssh exit8-vm --zone=asia-northeast3-a --project=think"
ssl_certificate_domains = tolist([
  "exit8-load-test.duckdns.org",
  "exit8-security-test.duckdns.org",
])
```

Terraform

- VPC, 서버넷, 방화벽, VM, Cloud SQL, Redis, LB를 코드로 선언적 관리
- PSA 구성으로 Cloud SQL / Redis를 퍼블릭 IP 없이 프라이빗 연결
- Google-managed SSL 인증서 + 헬스체크 자동 갱신
- terraform apply 한 번으로 전체 인프라 재현 가능

```
ansible
group_vars
all.yml
inventory
hosts.ini
roles
README.md
site.yml
```

```
# Project info
project_name: exit8
environment: demo

# Paths
app_dir: /opt/exit8
docker_compose_path: /opt/exit8/docker-compose.yml
backup_dir: /opt/exit8/backups

# VM specs
vm_memory_gb: 16
vm_cpu_cores: 4

# Database (Cloud SQL)
db_host: "{{ lookup('env', 'CLOUD_SQL_PRIVATE_IP') }}"
db_port: 5432
db_instance_name: exit8-postgres # Cloud SQL 인스턴스명 (Proxy)
db_name: exit8_app # PostgreSQL 데이터베이스명 (JD)
db_user: exit8_app_user
db_password: "{{ lookup('env', 'DB_PASSWORD') }}"

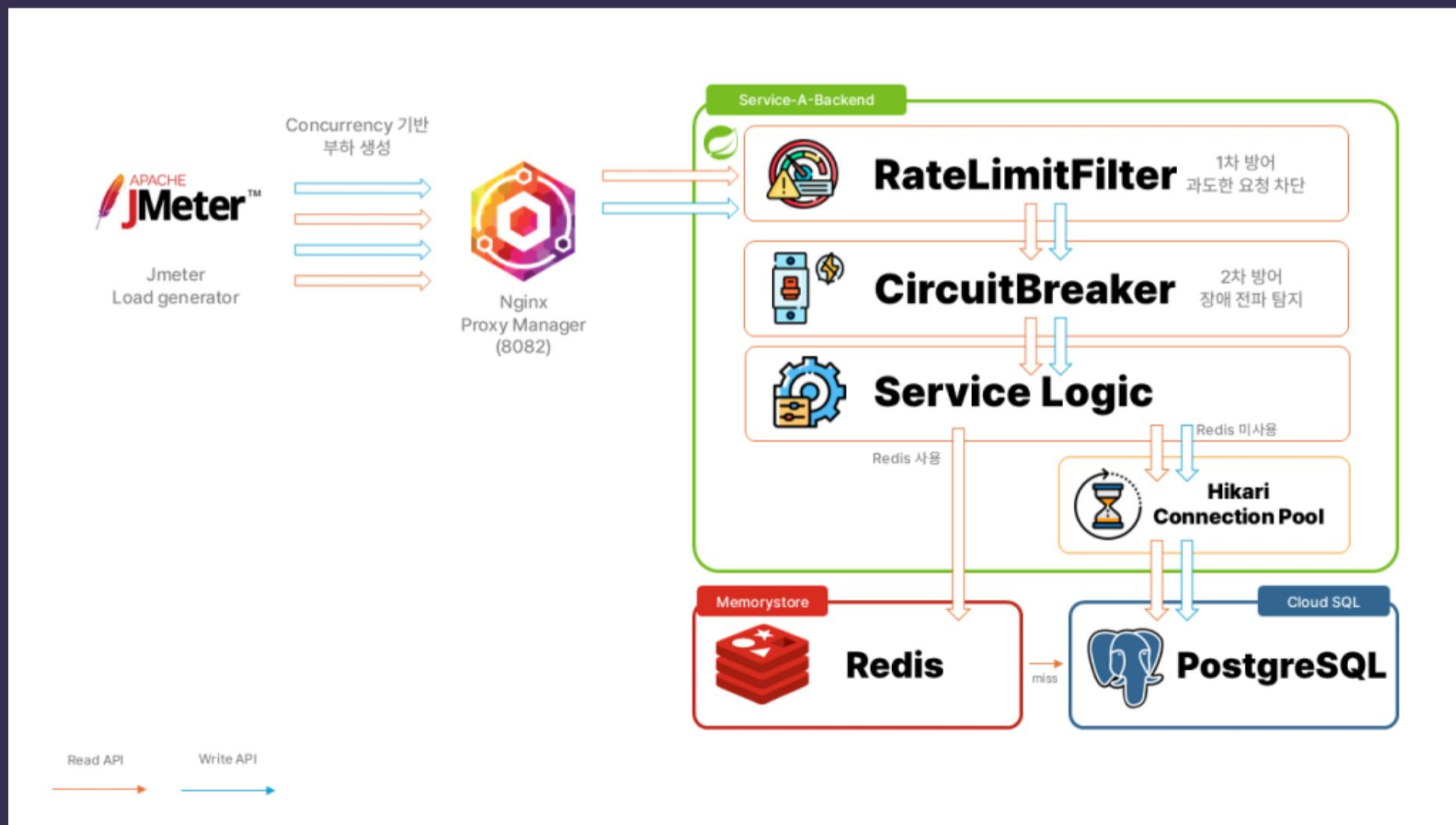
# Redis (Memorystore)
redis_host: "{{ lookup('env', 'REDIS_HOST') }}"
redis_port: 6379
```

Ansible

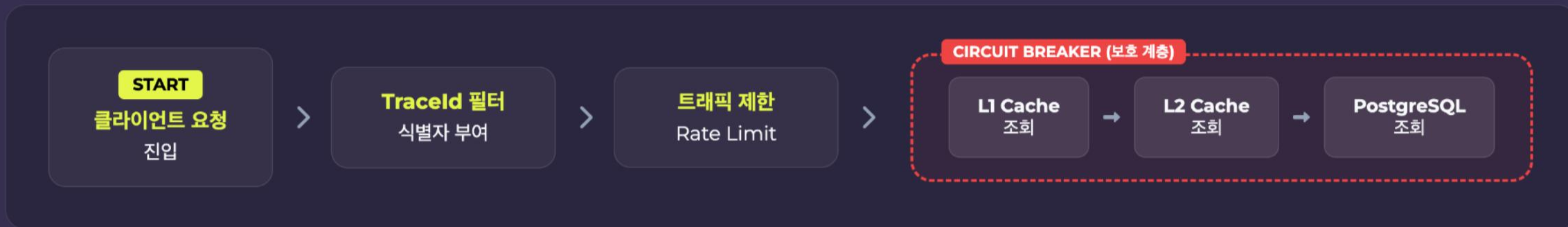
- VM 초기 설정 자동화: Docker 설치, 디렉토리 구성, .env 배포
- Docker Compose 스택 실행 및 컨테이너 헬스 확인까지 플레이북 1개
- Cloud SQL Proxy systemd 서비스 등록 자동화
- 단, 대상 VM이 1대뿐이라 멀티호스트 자동화라는 Ansible 본래 장점은 제한적 — 이 규모에선 오버엔지니어링 측면 있음

3 - 1. Scenario A

Service A 시나리오 흐름도



요청 처리 흐름 및 서비스 보호 구조



🛡️ Rate Limit — Bucket4j

구분	상세 설정
대상 엔드포인트	/api/load (IP 단위 제한)
용량 및 보충	Burst 40 / 초당 20개 보충
IP 캐시 전략	Caffeine (10분 만료)
제한 시 응답	HTTP 429 (Too Many Requests)

⚡ Circuit Breaker — Resilience4j

구분	상세 설정
보호 범위	Service 전체 로직 캡슐화
장애 임계치	최근 50회 중 실패율 50% 이상
차단 유지	Docker 10s / GCP 30s 유지
트리거 예외	DB Connection 관련 장애 한정

데이터 캐싱 및 데이터베이스 전략

L1: Caffeine

- 로컬 인메모리 캐시
- TTL: 300초 (Write 기준)
- 최대 1,000개 엔트리
- 지연시간: 1ms 미만

L2: Redis

- 글로벌 분산 캐시
- Key 접두사: exit8:
- Jackson JSON 직렬화
- 기본 캐시 매니저 설정

HikariCP

- Docker: 최대 50개
- GCP: 최대 100개
- Cloud SQL 총 200개 배분
- 일시적 연결 예외 대응

🕒 캐시 만료 정책 (TTL)

캐시 이름	만료 시간	주요 용도
loadTestResults	60초	실시간 부하 테스트 데이터
systemSnapshot	30초	시스템 상태 스냅샷
userSession	10분	사용자 인증 및 세션 정보
configData	1시간	시스템 설정 및 환경 변수
referenceData	24시간	정적 참조 및 코드 데이터

🗄️ 스토리지 역할 분리

구분	Redis (L2)	PostgreSQL (DB)
주요 역할	캐싱 및 세션 저장	영속성 데이터 관리
보안(SSL)	내부망 전용	GCP: Require 모드 강제
스키마 관리	-	GCP: Validate 모드
연결 방식	Lettuce 비동기	HikariCP 동기 방식
페이징	-	Slice (Count 쿼리 제거)

Service A 시나리오

```
=====
JMeter 부하 테스트 실행 설정
=====
1. Read 부하 테스트 (DB 조회)
2. Write 부하 테스트 (DB 쓰기)
실행할 테스트 유형을 선택하세요 (1 또는 2): 1

=====
Target 설정 (1: 로컬 / 2: 서버)
=====
접속 대상을 선택하세요 (1 또는 2): 2

공격 트래픽 동시 실행 스레드 수(ATTACK_THREADS)를 입력하세요 : 750

이 설정으로 몇 번 반복 실행할까요? (예 : 3): 3

풀 고갈 실험 모드를 활성화하시겠습니까? (y/n): n
[기본 풀 설정 사용]

=====
[반복 실행 설정]
- 테스트 유형 : read
- Target      : http://34.64.160.98:8082
- 공격 스레드 : 750
- 총 반복     : 3
- 반복 간 대기 : 20초
=====
```

Service-A Frontend — 실험 관측 대시보드

Rate Limit Filter 기반 IP 차단 현황 실시간 관측

Circuit Breaker

CLOSED

Rate Limit

OFF

click to toggle

Redis Cache

OFF

click to toggle

DB Pool

1/18 (6%)

Waiting Threads

0

0 (ok)

Avg. Response

1702ms

15시 58분 26초

Circuit Breaker / HTTP Distribution

15시 49분 49초

15시 58분 26초

200

100

429

0

500

0

503

0

Realtime Request Feed

polling 1~2s

Time	IP	Path	Status	Event
15시 58분 25초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
15시 58분 25초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
15시 58분 25초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
15시 58분 25초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
15시 58분 25초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
15시 58분 25초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
15시 58분 25초	10.10.10.10	/db-read	200	REQUEST_COMPLETED

IP Summary

10.10.10.10 Attack

Total94

Success (200)94 (100%)

Blocked (429)0 (0%)

Errors (5xx)0 (0%)

20.20.20.20 Normal

Total6

Success (200)6 (100%)

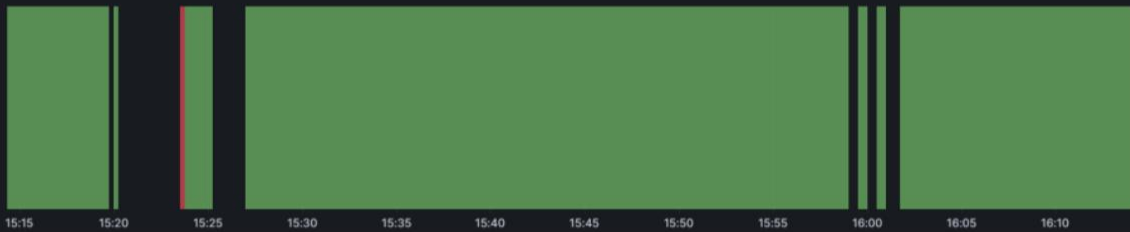
Blocked (429)0 (0%)

Errors (5xx)0 (0%)

Service A 시나리오

~ Row 1

Circuit Breaker State (testCircuit)



HTTP Status Distribution (Load APIs)



200
500

~ Row 2

HikariCP Connection Pool



Rate Limit (allowed / blocked)

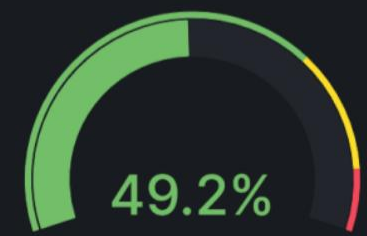


~ Row 3

Response Time (avg, Load APIs)



JVM Heap Usage (%)



기본 상태

Service-A Frontend — 실험 관측 대시보드

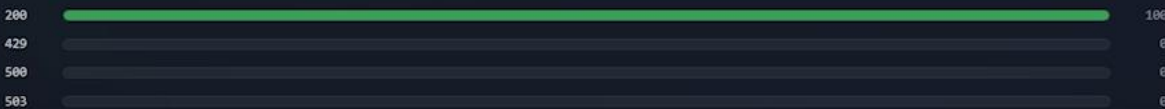
Rate Limit Filter 기반 IP 차단 현황 실시간 관측

Circuit Breaker **CLOSED** Rate Limit **OFF** Redis Cache **OFF** hit 98.0% DB Pool **0/100 (0%)** Waiting Threads **0** Avg. Response **210ms**

Circuit Breaker / HTTP Distribution

14시 41분 46초

14시 41분 52초



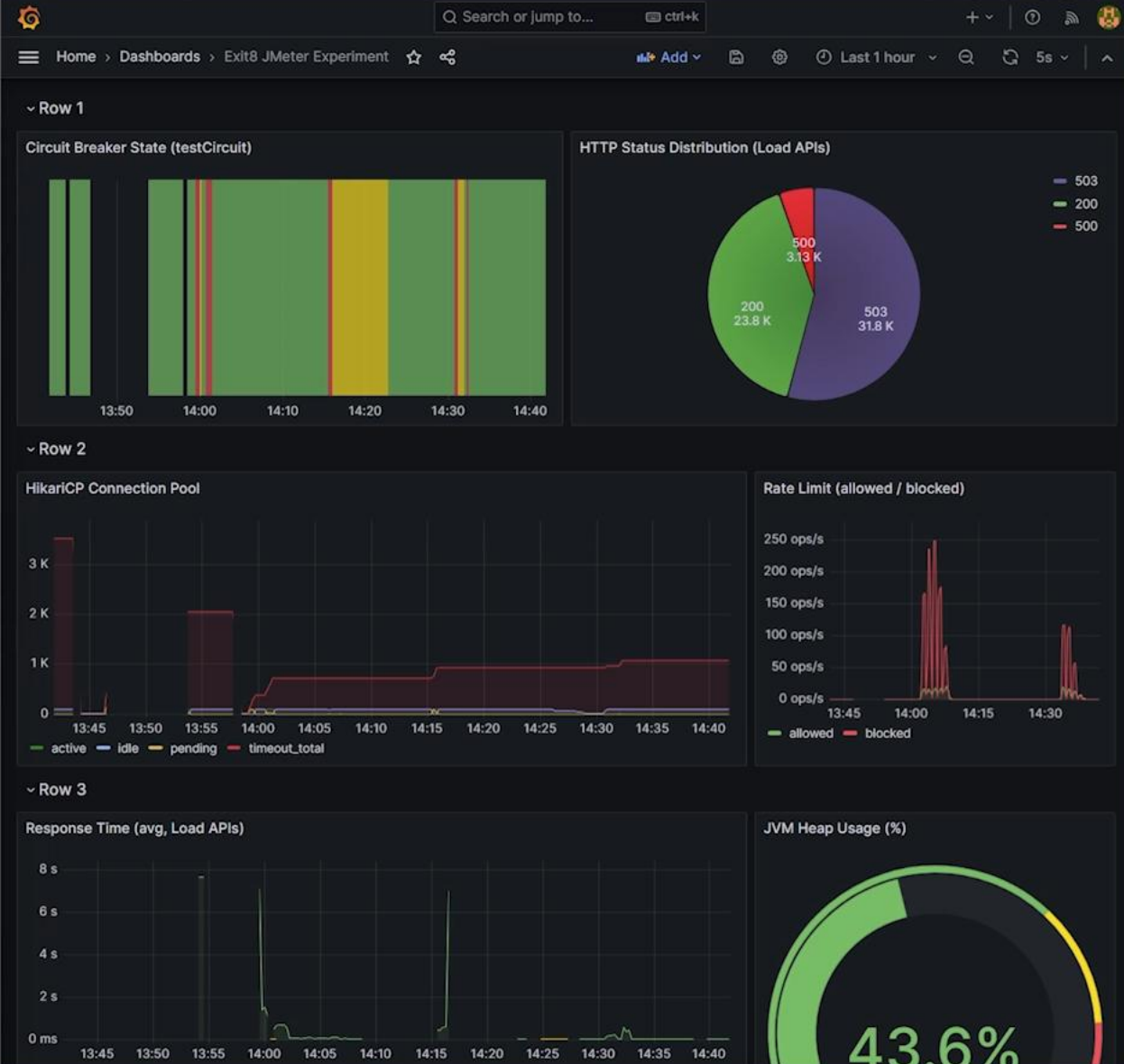
Realtime Request Feed

polling 1-2s

Time	IP	Path	Status	Event
14시 41분 1초	20.20.20.20	/db-read	200	REQUEST_COMPLETED
14시 41분 1초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED
14시 41분 0초	10.10.10.10	/db-read	200	REQUEST_COMPLETED

IP Summary

10.10.10.10	Attack
Total	96
Success (200)	96 (100%)
Blocked (429)	0 (0%)
Errors (500)	0 (0%)
20.20.20.20	Normal
Total	4
Success (200)	4 (100%)
Blocked (429)	0 (0%)
Errors (500)	0 (0%)



3 - 2. Scenario B

보안 취약점 분석, 관제, 알림 체계

ATTACKER

공격 요청
트래픽 유입

>

Service B

공격 대상

>

JSON Logs

로그 생성

>

Cloud Logging

로그 전달

>

Wazuh SIEM

이벤트 탐지

>

알림 발송

Email / Slack

🕒 시나리오 및 알림 목적

목표	상세 내용
취약점 재현	의도적으로 구현된 SQLi 패턴 등 공격 유입
탐지 규칙 검증	Wazuh의 수집/탐지 프로세스 유효성 확인
실시간 대응	임계치 초과 시 슬랙/메일 즉시 알림 체계 검증

</> 주요 분석 API

Endpoint	Role / Purpose
/api/v1/auth/login	SQL Injection 공격 및 인증 시도 대상
/api/v1/auth/register	테스트용 사용자 계정 생성
/health	컨테이너 및 서비스 가용성 체크
/profile	인증 성공 후 사용자 정보 조회

취약점 시나리오 및 관제 분석 포인트

👤 의도적 취약점 포인트

- **SQL Injection:** 문자열 결합 방식의 쿼리 구현
- **Auth Failure:** 반복적인 인증 실패 유도
- **IP Tracking:** X-Forwarded-For 헤더 기반 추적
- **Trace ID:** X-Trace-Id 연동을 통한 요청 추적

🔍 탐지 로그 이벤트 타입

- **SUSPICIOUS_INPUT:** SQLi 등 악성 패턴 감지
- **AUTH_UNAUTHORIZED:** 비정상 인증 실패
- **LOGIN_SUCCESS:** 최종 침투 성공 여부 확인
- **SYSTEM_ERROR:** 공격으로 인한 예외 발생

>_ JSON 로그 필드 구조

필드명	설명 및 역할
@timestamp	이벤트 발생 정확한 시각
level	로그 레벨 (INFO, WARN, ERROR)
message	공격 징후 및 상세 동작 메시지
ip / user_id	공격자 IP 주소 및 대상 계정 식별
mdc.trace_id	요청 생명 주기 추적용 고유 ID

🔍 실시간 관제 확인 포인트

[Priority 1] SQL Injection 시도
패턴 매칭을 통한 SUSPICIOUS_INPUT 로깅 확인

[Priority 2] Brute Force 탐지
반복되는 AUTH_UNAUTHORIZED 이벤트 임계치 검증

[Priority 3] 서버 예외 발생
SYSTEM_ERROR를 통한 서비스 가용성 침해 분석

Service B 시나리오



Exit8 Groupware

사내 문서 관리 시스템 로그인

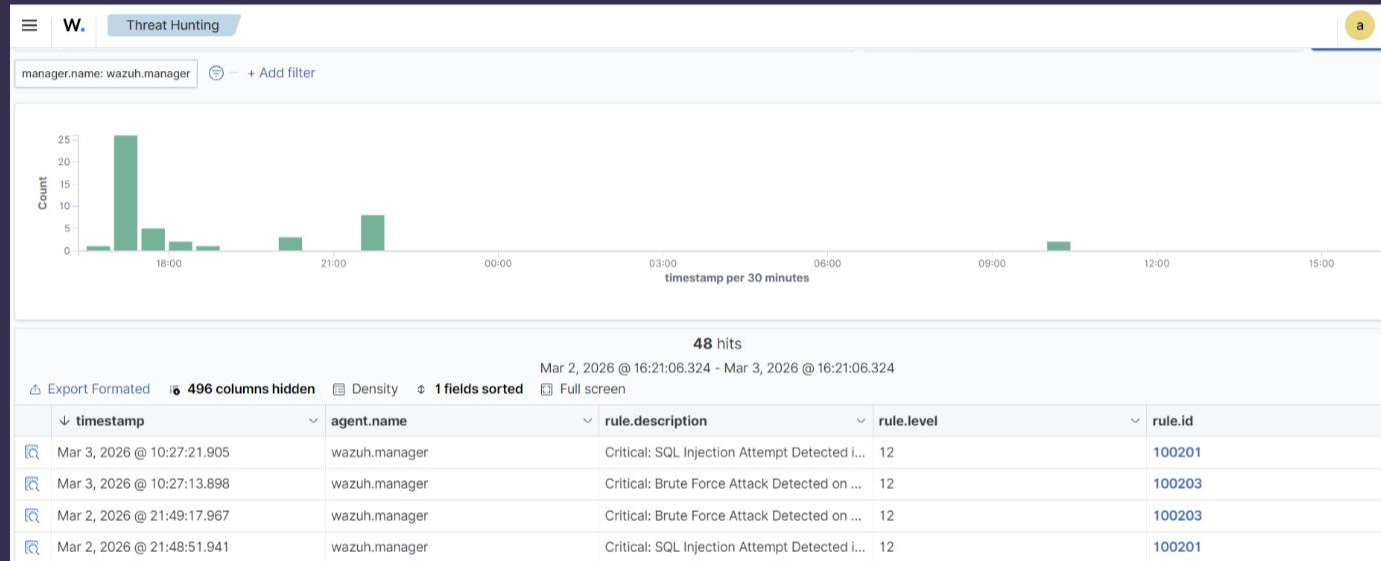
로그인

회원가입

```
> {
  insertId: "1uld4u3ffm58r1"
  > jsonPayload: {
    > container: {7}
    > instance: {3}
    message:
      "{@timestamp": "2026-03-03T17:08:33.835+09:00", "level": "WARNING", "mdc": {"trace_id": "e6a77ee0-e529-498a-ade7-2cf4a825cdb4"}, "user_id": null, "ip": "218.55.202.191", "message": "SUSPICIOUS_INPUT", "input_user": "ghghghzpz5'--", "patterns": "[ '--' ]", "type": "SQL_INJECTION_ATTEMPT"}"
    }
  logName: "projects/thinking-orb-485613-k3/logs/gcplogs-docker-driver"
  receiveTimestamp: "2026-03-03T08:08:34.844088925Z"
  > resource: {2}
  timestamp: "2026-03-03T08:08:33.836212281Z"
}
```

```
> * 2026-03-03 17:08:33.843 exit8-vm {@timestamp": "2026-03-03T17:08:33.842+09:00", "level": "INFO", "mdc": {"tra...
```

Service B 시나리오



☆ [Wazuh Alert - Level 12] Critical: SQL Injection Attempt Detected in Service-B Login (POST body)! 📧

보낸사람 hsj99316@gmail.com

받는사람 hsj99316@naver.com ghghghzpz5@naver.com

2026년 3월 3일 (화) 오전 10:27

🔄 영어 → 한국어 번역하기

Wazuh 보안 알람이 발생했습니다.

- Rule ID: 100201
- Alert Level: 12
- Description: Critical: SQL Injection Attempt Detected in Service-B Login (POST body)!

[Log Details]

```
{ "@timestamp": "2026-03-03T10:27:20.410+09:00", "level": "WARNING", "mdc": { "trace_id": "2039e078-8c39-4b81-ae9b-962281c1a336" }, "user_id": null, "ip": "211.112.173.38", "message": "SUSPICIOUS_INPUT", "input_user": "ghghghzpz5", "patterns": [ "-- ], "type": "SQL_INJECTION_ATTEMPT" }
```

☆ [Wazuh Alert - Level 12] Critical: Brute Force Attack Detected on Service-B Login! 📧

보낸사람 hsj99316@gmail.com

받는사람 hsj99316@naver.com ghghghzpz5@naver.com

2026년 3월 3일 (화) 오전 10:27

🔄 영어 → 한국어 번역하기

Wazuh 보안 알람이 발생했습니다.

- Rule ID: 100203
- Alert Level: 12
- Description: Critical: Brute Force Attack Detected on Service-B Login!

[Log Details]

```
{ "@timestamp": "2026-03-03T10:27:13.758+09:00", "level": "WARNING", "mdc": { "trace_id": "9ee94d0d-b970-4124-92a0-d155c6f32de3" }, "user_id": null, "ip": "211.112.173.38", "message": "AUTH_UNAUTHORIZED", "input_user": "ghghghzpz5", "success": false }
```



Exit8 Groupware

사내 문서 관리 시스템 로그인

로그인

회원가입

로그인 정보가 일치하지 않습니다.

4. Backup Retention & Estimated Cost

데이터 백업 및 가용성 전략

Cloud SQL (PostgreSQL 15)

항목	설정 및 정책
백업 주기	매일 02:00 UTC (자동 백업)
백업 보관	최근 14일 데이터 유지
PITR (시점 복구)	최대 7일 이내 임의 시점 복구 지원
연결 보안	TLS 암호화 강제 (ENCRYPTED_ONLY)
가용성 모델	ZONAL (비용 최적화 단일 존 구성)
자동 확장	10GB SSD 시작 (부하 시 자동 증설)

백업 관리 핵심: 데이터 무결성을 위해 Zonal 인스턴스임에도 불구하고 2주간의 충분한 백업 보관 및 시점 복구(PITR) 기능을 활성화하여 운영 안정성을 확보하였습니다.

Redis (Memorystore) 정책

"의도적인 백업 미적용"

- Redis는 L2 캐시 전용 휘발성 저장소
- 재시작 시 앱이 DB에서 데이터를 자동 재로딩
- Source of Truth:** PostgreSQL이 모든 원본 보관

> 복구 명령어 (PITR Clone)

```
gcloud sql instances clone exit8-postgres \
  exit8-postgres-restored \
  --point-in-time="2026-03-01T10:00:00Z"
```

로그 아카이브 및 모니터링 임계치



로그 발생



VM 로컬 (7일)



GCS 아카이브



영구 삭제

로그 유형별 상세 보관 정책

로그 유형	VM 로컬	GCS 보관	스토리지 클래스 전환
앱 컨테이너 로그	Cloud Logging	30일	자동 관리
Nginx / 시스템	7일	30일	1일 후 NEARLINE
Cloud SQL 에러	—	1년	30일(NL) → 90일(CL)
LB 5xx 에러	—	1년	30일(NL) → 90일(CL)
Wazuh (보안 로그)	7일	180일	7일(NL) → 30일(CL)

모니터링 알림 임계값

항목	임계값	감지
VM CPU 사용률	> 80%	5분
DB 활성 연결	> 160 (80%)	5분
Redis 메모리	> 80%	5분
LB 5xx 에러율	> 5 req/s	5분

GCP 서비스 스펙

서비스명	세부 스펙	주요 역할
Compute Engine	e2-standard-4 (4vCPU / 16GB RAM / 100GB SSD)	NPM, Docker 컨테이너 및 전체 서비스 호스팅
Cloud SQL	db-custom-2-8192 (2vCPU / 8GB / 10GB SSD)	PostgreSQL 15 운영 (Max 200 Conn)
Memorystore	Redis 7.0, 1GB BASIC Tier	L2 분산 캐시 저장소
HTTPS Load Balancer	글로벌, TLS 1.2+, Google 관리형 인증서	트래픽 분산 및 SSL Termination
GCS 버킷 (2개)	asia-northeast3 리전	에러 로그 및 시스템 로그 장기 보관
Secret Manager	3개 시크릿 (DB, Grafana, Wazuh)	민감 정보 중앙 관리 및 주입

월간 예상 운영 비용 분석

서비스별 월 비용 상세

항목	예상 비용 (월)
Compute Engine (e2-standard-4)	~\$70.00
Cloud SQL (db-custom-2-8192)	~\$50.00
Memorystore Redis (1GB BASIC)	~\$35.00
HTTPS Load Balancer	~\$20.00
GCS 로그 아카이브	~\$0.15
월간 합계 (Total Estimated)	~\$175.15 / 월

비용 최적화 포인트

- ZONAL SQL:** 가용성 모델 최적화로 50% 절감
- BASIC Redis:** 캐시 용도에 맞는 최소 비용 선택
- Storage Lifecycle:** 자동 전환으로 저장료 최소화
- Cloud Armor:** 필요한 정책만 선별 적용

주간 환산: 약 \$44

(약 ₩65,000 / 주)

월간 합계 약 ₩252,000 수준으로 고성능 보안 인프라를 효율적으로 운영합니다.

5. Troubleshooting & Evaluation

트러블슈팅

INFRASTRUCTURE

📦 2중 프록시 리다이렉트 루프

증상: LB 헬스 체크 실패 및 "no healthy upstream" 오류 발생
원인: GCP LB가 SSL 종료 후 HTTP로 전달하는데, 내부 NPM에서 HTTPS 리다이렉트를 시도하여 무한 루프 형성

```
# services/npm/custom/http_redirect.conf
return 301 https://$host$request_uri;
location /health { return 200 "OK"; }
```

* 최적화: 유입(HTTPS) → LB(SSL Termination) → VM:80(HTTP) → NPM 수신

SECURITY SIEM

🛡️ Wazuh 통합 운영 및 최적화

증상

원인 및 조치

Dashboard 접속 불가

.env API 키 특수문자(# → !) 불일치 현상 수정

블룸 충돌 오류

기존 설정과 신규 설정 충돌 → 특정 블룸 선택 삭제 후 재빌드

탐지 노이즈 발생

Timeframe 조정(120s→60s) 및 알림 레벨 최적화 (10→3)

```
docker volume rm $(docker volume ls -q | grep -E
"wazuh_api|wazuh-dashboard")
```

트러블슈팅

BACKEND-A (JAVA/SPRING)

⚙️ Build Environment & CI/CD

이슈: Gradle Wrapper JAR가 .gitignore 설정 오류로 누락되어 CI 빌드 실패

```
# .gitignore 표준화
!gradle/wrapper/gradle-wrapper.jar
!gradle/wrapper/gradle-wrapper.properties
```

- PR 체크리스트에 Wrapper 포함 여부 필수 항목 추가
- CI 단계에서 Wrapper 기반 빌드 검증 프로세스 고정

📊 Performance Metric Standards

이슈: JMeter 테스트 시 Concurrency(동시성)와 Throughput(처리량) 기준 혼용으로 결과 해석 오류

구분	판정 기준 통일
부하 조건	Concurrency (Number of Threads) 기준 설정
결과 판정	Throughput (RPS) 기준으로 성능 지표 통일

* 부하 가해량과 실제 처리량을 명확히 분리하여 정확한 P95 응답 시간 및 성능 한계점 분석 완료

트러블슈팅

BACKEND-B (PYTHON/FASTAPI)

🔗 SQL Injection 오탐 및 환경 개선

정상적인 입력 내 작은따옴표(')가 공격으로 오탐되어 SIEM 알람 폭증

```
# Before: 모든 작은따옴표 탐지
re.compile("'", re.IGNORECASE)

# After: SQL 문법적 의미가 있는 패턴만 탐지
re.compile("'\\s*(?:OR|AND)\\b", re.IGNORECASE)
```

- ✅ **Alembic 도입:** DB 스키마 변경 미반영 문제 자동화 해결
- ✅ **Trace ID 동기화:** 분산 추적 불일치 원인 제거

FRONTEND & LOAD BALANCER

🕒 504 Gateway Timeout 해결

증상: 고부하 상황에서 P95 응답시간 지연 시 504 에러 발생
원인: GCP LB 기본 타임아웃(30s)이 백엔드 처리 시간보다 짧음

```
# Nginx & Terraform 타임아웃 동기화 (30s → 100s)
proxy_read_timeout 100s;
# Terraform Backend Service
timeout_sec = 100
```

* 결과: LB ↔ Nginx ↔ Backend 전 구간 임계치 일치로 연결 안정성 확보

Troubleshooting Summary: 인프라 설정 오류부터 애플리케이션 로직 오탐까지, 실제 운영 환경에서 발생 가능한 시나리오를 해결하며 인프라의 가용성과 탐지 정확도를 검증했습니다.

자체 평가 의견 (SELF-EVALUATION)

기획 달성도

IaC 기반 인프라 구축부터 보안 탐지·모니터링·CI/CD 자동화까지 목표 전 영역 구현 완료.

Cloud Armor 미활성
(쿼터 제한)을 제외하면 기획 대비 **95%** 이상 달성.

핵심 성과

- 해킹 시도 **2~3초** 내 Wazuh 알람 체계 구축
- 스레드 1,000개 부하에서 **115 RPS** 내구성 확인
- IP Rate Limit + Circuit Breaker로 **장애 전파 최소화**.

향후 개선점

- 실시간 Risk Scoring: IP·계정별 실패 횟수 기반 대시보드 구현
- DB 고가용성 확보 및 프론트 실시간성 최적화
- 슬랙·디스코드 알람 다중화 및 Cloud Armor 적용
- 클라우드 비용 최적화 및 K8s 기반 Blue-Green 무중단 배포 도입

QUESTIONS?

경청해 주셔서 감사합니다. 질문에 성실히 답변 드리겠습니다.

Thank You

Github Repo: <https://github.com/exit8-ktcloud/self-managed-infrastructure>

시연영상 Link: <https://youtu.be/YXcl1FrCYiw>